

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

Sommersemester 2026



Eine kleine Spielumgebung als Untersuchungs-labor schaffen

„Every act of creation is first an act of destruction.“
(Pablo Picasso)

PD Prof. Dr. Marco Block-Berlitz

Einfache Spielumgebung als Untersuchungslabor

Mit einer sehr einfachen Umsetzung einer Spielwelt lassen sich bereits einige wichtige Aspekte der Computergrafik im Bereich visueller Simulationen erläutern.

Typische Elemente

- Debug-Umgebung
- Statische, dynamische Objekte
- Kapselung von Verhalten
- Objektmanager
- Steuerungsverhalten
- Hinderniserkennung
- Schwarmverhalten
- Wassereffekt
- ...

spielumgebung/korallenriff/debug

Statische und dynamische Spielelemente

Wenn wir eine Spielumgebung anschauen, gibt es typischerweise **statische** Objekte ohne Bewegung und **dynamische** Objekte mit Bewegung. Bäume, Gebäude oder andere Elemente werden in einer Szene platziert und bewegen sich nicht. Hier lassen wir mal kurz ausser vor, dass sich bei einem Spieleroller natürlich die Gesamtszene kontinuierlich bewegt. In diesem Beispiel stellen die Spielfiguren bewegliche Objekte dar.

Gemeinsamkeiten zusammenfassen

abstrakt
BaseObject
ist ab
Post xPos, yPos
abstrakt render()

11

5

Ziele und Inhalt

- Einfache Spielumgebung als Untersuchungslabor
- Der perfekte Flummi – die Minimallösung
- Statische und dynamische Spielelemente
- Das abstrakte BasisObjekt
- Kreis erbt von BasisObjekt
- Welt eines einsamen Kreises
- BeweglichesObjekt erweitert BasisObjekt
- Flummi erbt von BeweglichesObjekt
- Welt eines einsamen Flummies
- Objektmanager über HashMap
- Welt der bunten Flummies
- Strategisch mit Verhalten umgehen
- Interface Verhalten
- Komposition in BeweglichesObjekt unterbringen
- Verhalten ganz einfach erweitern
- Zufällige Richtungsbewegung realisieren
- Welt der bunten Flummies erweitern
- Programmbeispiele in Eclipse ausprobieren
- Welt der bunten Flummis in Processing
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben





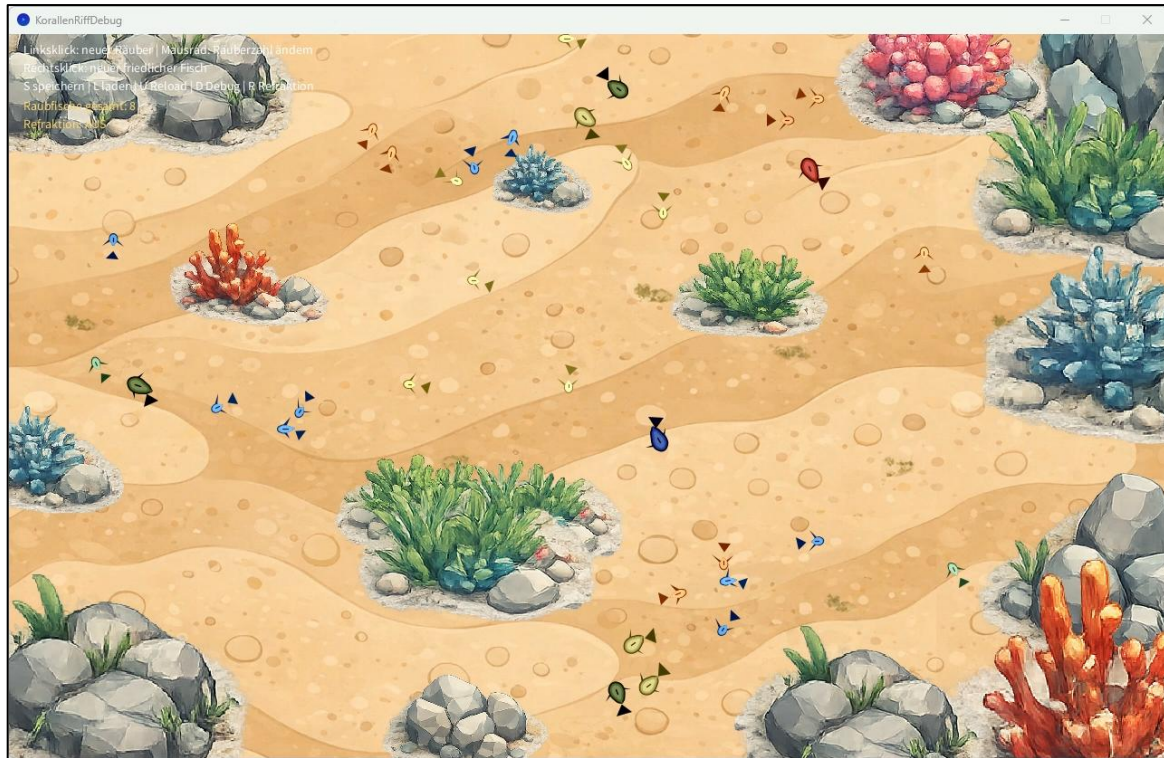
Eine einfache Spieleumgebung als **Labor** für
Partikelsimulationen, Schwarmverhalten und
weitere Effekte

Einfache Spielumgebung als Untersuchungslabor

Mit einer sehr einfachen Umsetzung einer Spielwelt lassen sich bereits einige wichtige **Aspekte der Computergrafik** im Bereich visueller Simulationen erläutern.

Typische Elemente

- Debug-Umgebung
- Statische, dynamische Objekte
- Kapselung von Verhalten
- Objektmanager
- Steuerungsverhalten
- Hinderniserkennung
- Schwarmverhalten
- Wassereffekt
- ...

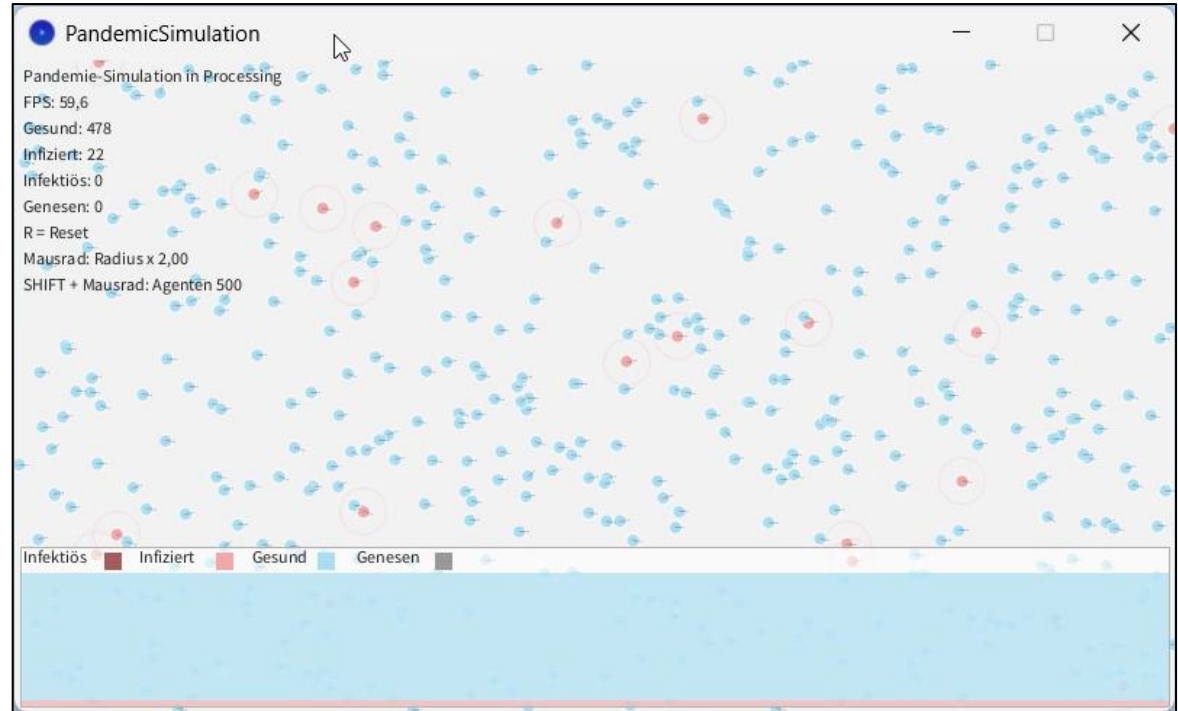


Einfache Spielumgebung als Untersuchungslabor

Darauf aufbauend lassen sich auch Simulationen wie diese erzeugen. Hier sehen wir eine [Pandemie-Simulation](#).

Untersuchen lassen sich

- Ansteckungsszenarien
- Abstandsregeln
- Inkubationszeiten
- ...

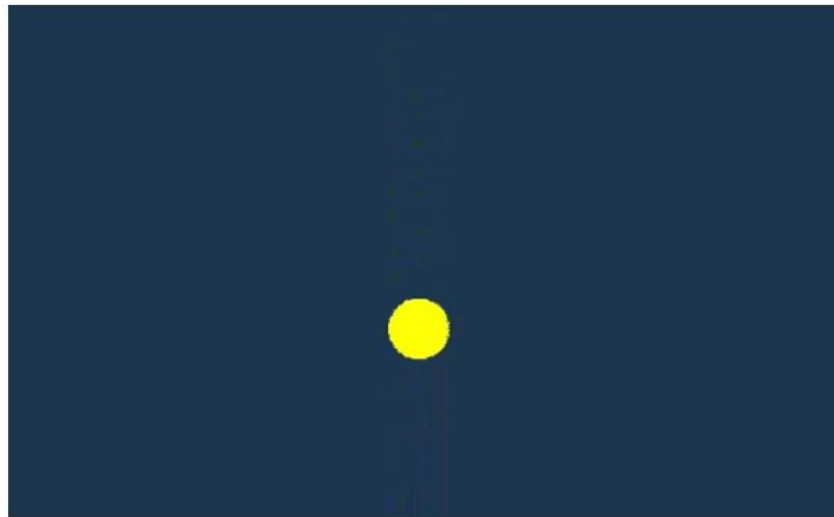
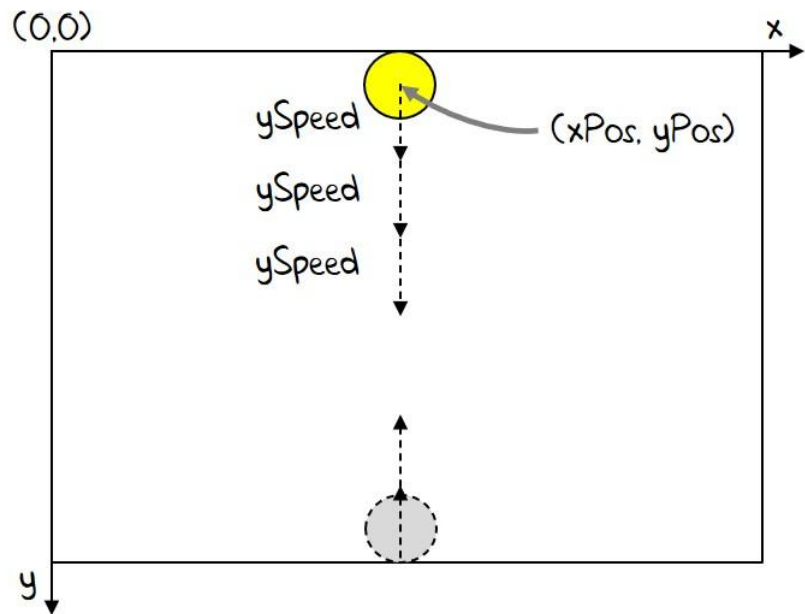




Wir wollen mit einem kleinen **Flummi** beginnen.

Der perfekte Flummi

Wir beginnen mit einer kleinen Aufgabe, der Erzeugung eines perfekten Flummis, die wir zunächst straightforward lösen wollen. Anschließend werden wir die Lösung immer mehr anpassen und sie auf diese Weise zu einer kleinen Spielumgebung mit verschiedenen Fazetten umformulieren.

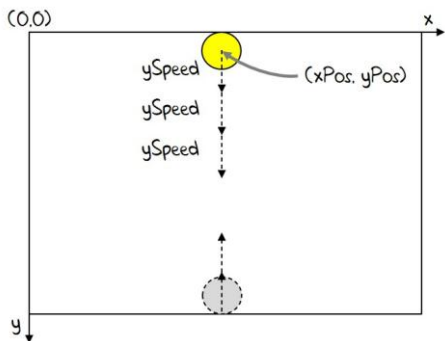


Die Minimallösung

Wenn wir das zielgerichtet implementieren, ohne jegliche Generalisierung, Einhaltung korrekter Physikregeln oder Erweiterbarkeit zu betrachten, könnte die Lösung so aussehen:

```
public class PerfekterFlummi extends LWJGLBasisFenster {
    private float xPos, yPos, ySpeed;

    public PerfekterFlummi() {
        super("Perfekter Flummi", 640, 480);
        xPos = 320;
        yPos = 0;
        ySpeed = 1.5f;
    }
}
```



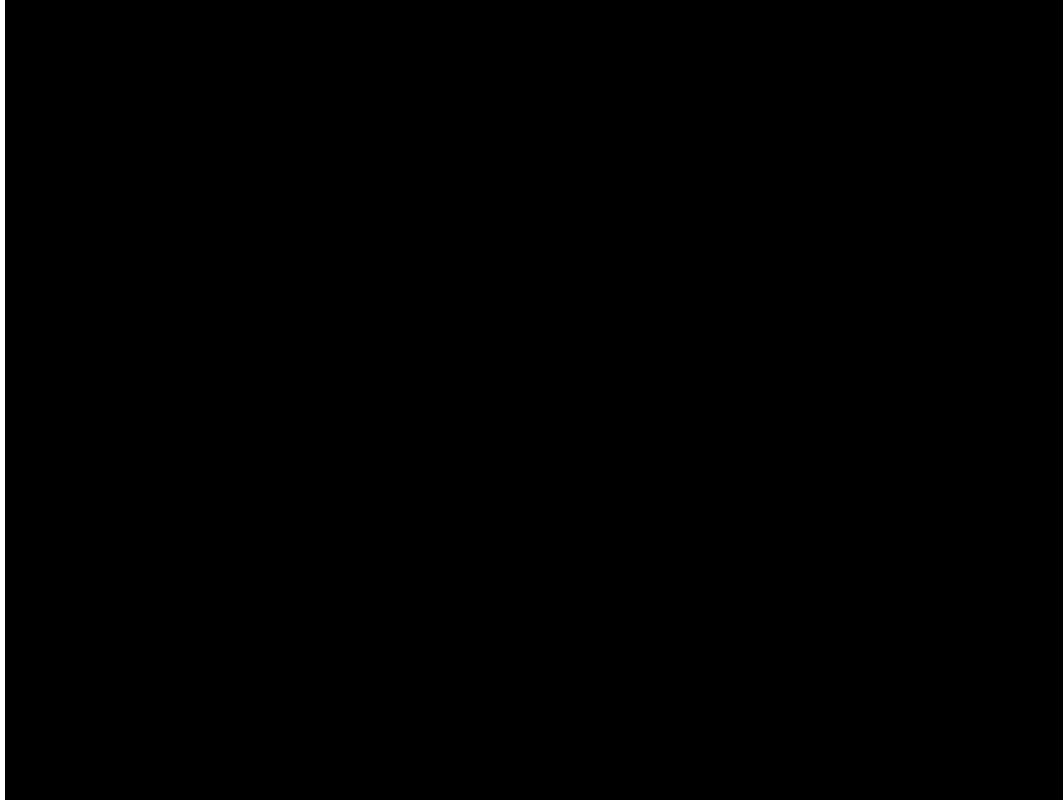
```
@Override
public void renderLoop() {
    while (!Display.isCloseRequested()) {
        // Bildschirm löschen, OpenGL vorbereiten
        // gelben Ball an (xPos, yPos) zeichnen
        ...

        // Position in Abhängigkeit zur Geschwindigkeit
        yPos = yPos + ySpeed;

        // an den Rändern Richtung umkehren
        if (yPos > 480 || yPos < 0)
            ySpeed = ySpeed * -1;

        Display.update();
    }
}
```

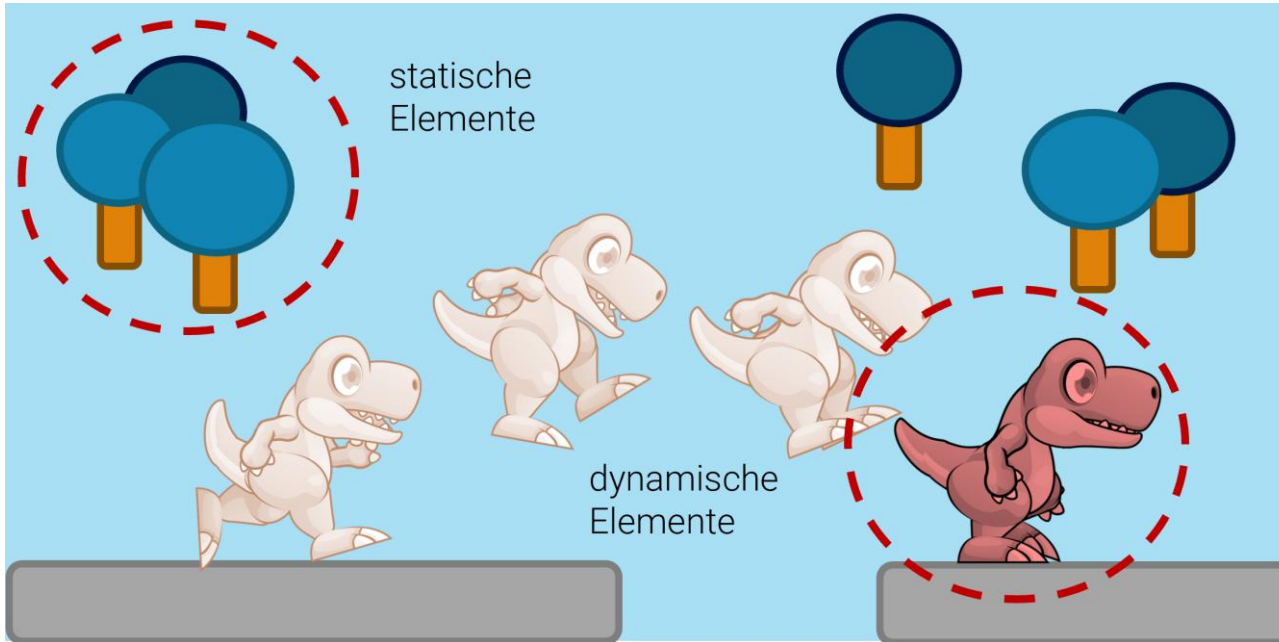
Minimallösung: der perfekte Flummi



Das Programm tut das Gewünschte.

Statische und dynamische Spielelemente

Wenn wir uns Spielumgebungen anschauen, gibt es typischerweise **statische Objekte** ohne Bewegung und **dynamische Objekte** mit Bewegung. Bäume, Gebäude oder andere Elemente werden in einer Szene platziert und bewegen sich nicht. Hier lassen wir mal kurz außen vor, dass sich bei einem Sidescroller natürlich die Gesamtscene kontinuierlich bewegt. In diesem Beispiel stellen die Spielfiguren bewegliche Objekte dar.



Gemeinsamkeiten
zusammenfassen:

<code>abstract BasisObjekt</code>
<code>int id float xPos, yPos</code>
<code>abstract render()</code>

Das abstrakte BasisObjekt

Die Gemeinsamkeiten und Unterschiede wollen wir über Vererbung ausdrücken. Zunächst erzeugen wir eine Klasse **BasisObjekt**, in der jedem Objekt eine eindeutige ID und eine Position zugewiesen wird. Da wir uns zunächst auf die Umsetzung unseres einfachen Beispiels konzentrieren wollen, belassen wir es zunächst bei den Parametern **xPos** und **yPos**. Damit jedes Objekt in individueller Weise visualisiert wird, machen wir aus der Klasse eine abstrakte Klasse und fordern die Implementierung einer abstrakten Methode **render**.

```
public abstract class BasisObjekt {  
    public int    id;  
    public float xPos, yPos;  
  
    public BasisObjekt() {  
        this(0, 0);  
    }  
  
    public BasisObjekt(float xPos, float yPos) {  
        this.xPos = xPos;  
        this.yPos = yPos;  
    }  
  
    public abstract void render();  
}
```

abstract BasisObjekt
int id float xPos, yPos
abstract render()

Der Kreis erbt von BasisObjekt

Schauen wir uns an, wie wir von dem **BasisObjekt** ableiten können, um einen gelben Flummi in OpenGL zu implementieren.

```
public class Kreis extends BasisObjekt {
    private float radius;
    private float r, g, b;

    public Kreis() {
        this(0, 0, 20.0f, 1, 1, 0);
    }

    public Kreis(float xPos, float yPos) {
        this(xPos, yPos, 20.0f, 1, 1, 0);
    }

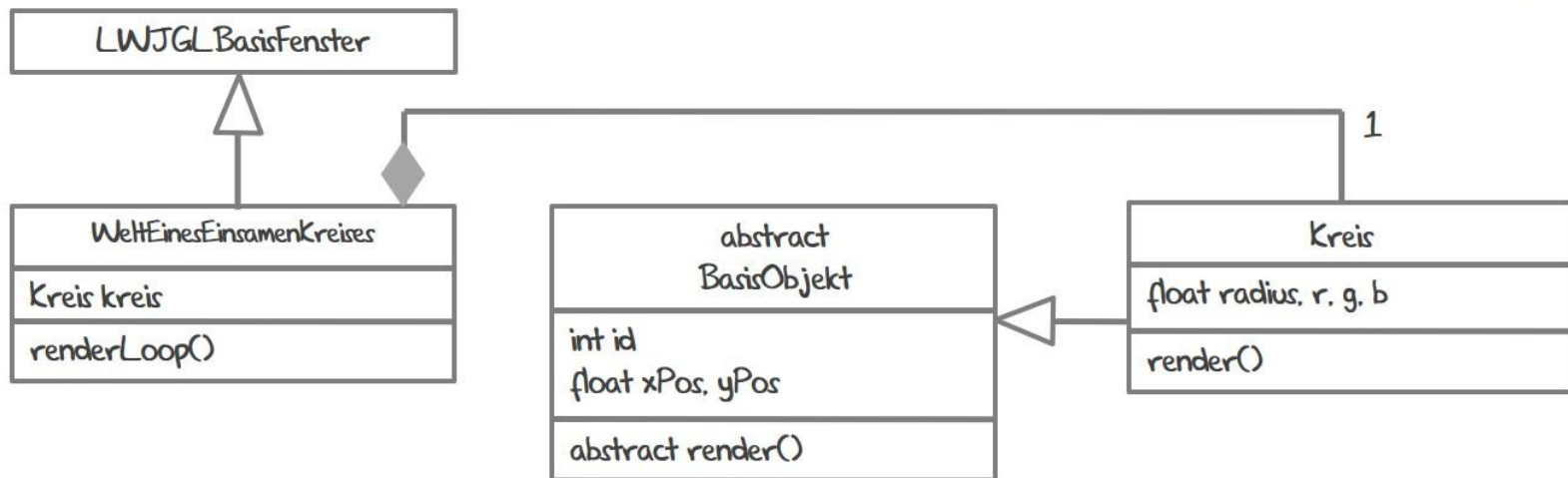
    public Kreis(float xPos, float yPos, float radius,
                float r, float g, float b) {
        super(xPos, yPos);
        this.radius = radius;
        this.r=r;
        this.g=g;
        this.b=b;
    }
}
```

```
@Override
public void render() {
    glColor3d(r, g, b);
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(xPos, yPos);
    for (int angle = 0; angle <= 360; angle += 5) {
        double rad = Math.toRadians(angle);
        glVertex2f(
            xPos + (float) Math.cos(rad) * radius,
            yPos + (float) Math.sin(rad) * radius
        );
    }
    glEnd();
}
```

Welt eines einsamen Kreises

Der **Kreis** erbt von Basisobjekt und implementiert die Methode **render**. Weiterhin wird die Spielumgebung **WeltEinesEinsamenKreises** erzeugt und dort ein **Kreis** platziert. Wir lesen die Komposition als: Die **WeltEinesEinsamenKreises** „hat einen“ **Kreis**.

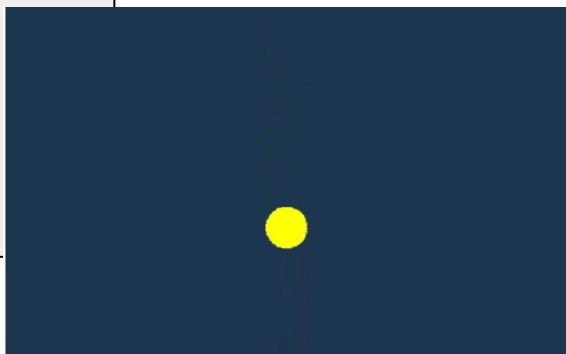
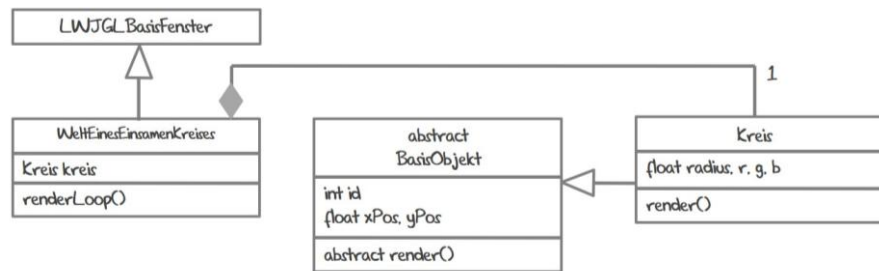
Erster Visualisierungsentwurf



Welt eines einsamen Kreises

Hier sehen wir, wie in der Welt zunächst ein einzelner **Kreis** verwaltet und im **renderLoop** gezeichnet wird. Das Ergebnis ist nicht besonders spektakulär, aber stellt einen guten Startpunkt für weitere Überlegungen dar.

```
public class WeltEinesEinsamenKreises extends LWJGLBasisFenster {  
    private Kreis kreis;  
  
    public WeltEinesEinsamenKreises() {  
        super("Welt eines einsamen Kreises", 640, 480);  
        kreis = new Kreis(320, 240);  
    }  
  
    @Override  
    public void renderLoop() {  
        while (!Display.isCloseRequested()) {  
            loescheBildschirm();  
            kreis.render();  
            Display.update();  
        }  
    }  
}
```



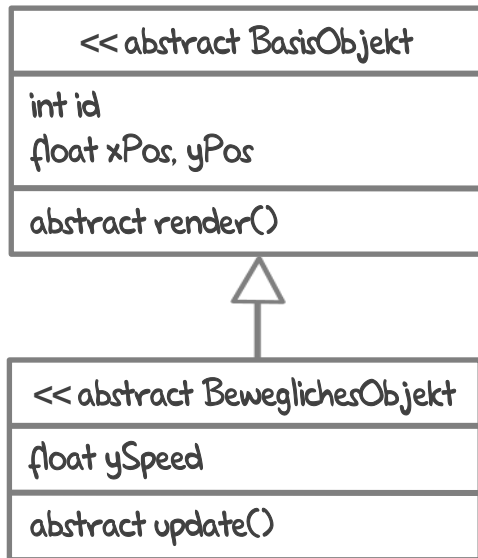


Jetzt wollen wir **Bewegung** ins Spiel bringen ...

BeweglichesObjekt erweitert BasisObjekt

Mit der Klasse **BasisObjekt** können wir Objekte eindeutig identifizieren, positionieren und diese mit **render** zeichnen. Für bewegliche Objekte wollen wir zunächst bei unserem Beispiel mit dem perfekten Flummi bleiben und den Parameter **ySpeed** einfügen. Weiterhin brauchen wir eine Methode **update**, die sich um die Änderung der Position bzw. den Richtungswechsel am Weltrand kümmern soll:

```
public abstract class BeweglichesObjekt extends BasisObjekt {  
    public float ySpeed;  
  
    public BeweglichesObjekt(float xPos, float yPos, float ySpeed) {  
        super(xPos, yPos);  
        this.ySpeed = ySpeed;  
    }  
  
    public abstract void update();  
}
```



Flummi erbt von BeweglichesObjekt

Von der Klasse **BasisObjekt** werden wir einen Flummi analog zum Kreis ableiten und mit dem entsprechenden Leben füllen.

```
public class Flummi extends BeweglichesObjekt {
    private static int objCounter = 0;
    private float radius;
    private float r, g, b;

    public Flummi() {
        this(0, 0, 2, 20.0f, 1, 1, 0);
    }

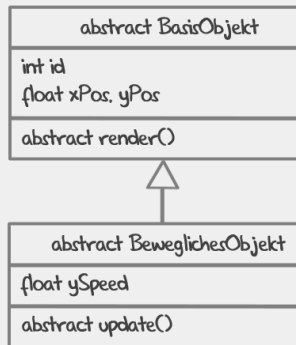
    public Flummi(float xPos, float yPos, float ySpeed) {
        this(xPos, yPos, ySpeed, 20.0f, 1, 1, 0);
    }

    public Flummi(float xPos, float yPos, float ySpeed, float radius,
        float r, float g, float b) {
        super(xPos, yPos, ySpeed);
        this.radius = radius;
        this.r=r;
        this.g=g;
        this.b=b;
        this.id = ++objCounter;
    }
}
```

```
@Override
public void update() {
    yPos=yPos+ySpeed;

    if (yPos>480 || yPos<0)
        ySpeed = ySpeed * -1;
}

@Override
public void render() {
    glColor3d(r, g, b);
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(xPos, yPos);
    for (int angle=0; angle<360; angle+=5) {
        glVertex2f(xPos + (float)Math.sin(angle) * radius,
            yPos + (float)Math.cos(angle) * radius);
    }
    glEnd();
}
}
```



Welt eines einsamen Flummies

Unsere erweiterte Version verwaltet einen Flummi, der im **renderLoop** gezeichnet wird und bereits wieder über eine vertikale Bewegung verfügt.

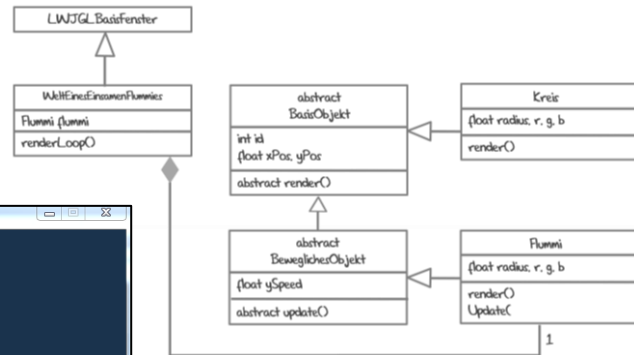
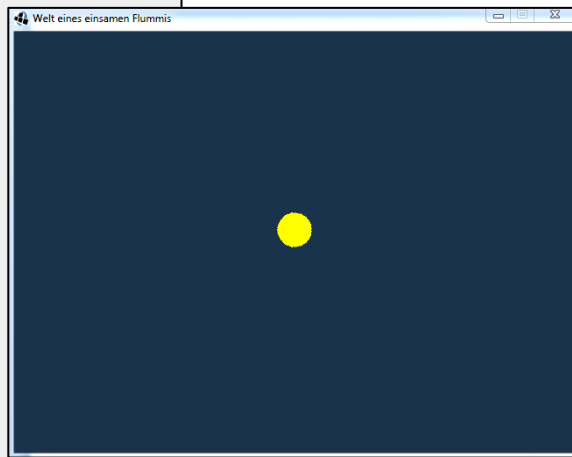
```
public class WeltEinesEinsamenFlummis extends LWJGLBasisFenster {
    private Flummi flummi;

    public WeltEinesEinsamenFlummis() {
        super("Welt eines einsamen Flummis", 640, 480);
        flummi = new Flummi(320, 240, 2);
    }

    @Override
    public void renderLoop() {
        while (!Display.isCloseRequested()) {
            loescheBildschirm();

            flummi.update(); // Physik updaten
            flummi.render(); // einen Flummi anzeigen

            Display.update();
        }
    }
}
```





Wir haben den Wunsch viele unterschiedliche
Flummies zu verwenden.

Wie **verwalten** wir jetzt **viele Objekte**?

Einen Objektmanager anlegen

Jetzt wird auch klar, dass es sinnvoll war, jedem Objekt eine ID zu geben. Mit Hilfe einer [HashMap](#) können wir Objekte registrieren und verwalten. Der Objektmanager sollte als [Singleton](#) implementiert werden [1].

```
package kapitel01.particle_version03_objectmanager;
import java.util.HashMap;

public class ObjektManager {
    private HashMap<Integer, Flummi> flummies;

    // *****
    // ObjektManager als Singleton realisieren
    private static ObjektManager exemplar = new ObjektManager();

    private ObjektManager() {
        flummies = new HashMap<Integer, Flummi>();
    }

    public static ObjektManager getExemplar() {
        return exemplar;
    }

    public Object clone() throws CloneNotSupportedException {
        throw new CloneNotSupportedException("Clonen ist nicht erlaubt");
    }

    // *****
}
```

```
public void registriereFlummi(Flummi obj) {
    flummies.put(obj.id, obj);
}

public void entferneFlummi(Flummi obj) {
    flummies.remove(obj.id);
}

public Flummi getFlummi(int objID) {
    return flummies.get(objID);
}

public HashMap<Integer, Flummi> getFlummiMap() {
    return flummies;
}

public int getFlummiSize() {
    return flummies.size();
}
}
```

Welt der bunten Flummies

Wir können in unserer Welt jetzt die **Erzeugung und Verwaltung vieler Objekte** voranstellen und damit sehr schnell bspw. die Komplexität einer Partikelsimulation verwalten. In unserem nächsten Beispiel werden 500 Flummies erzeugt, im Objektmanager verwaltet und in einem Schleifendurchlauf im Renderloop gezeichnet.

```
public class WeltDerBuntenFlummis extends LWJGLBasisFenster {
    private ObjektManager flummies;

    public WeltDerBuntenFlummis() {
        super("Welt der Flummies", 640, 480);
        initDisplay();
        flummies = ObjektManager.getExemplar();
        erzeugeFlummies(500);
    }

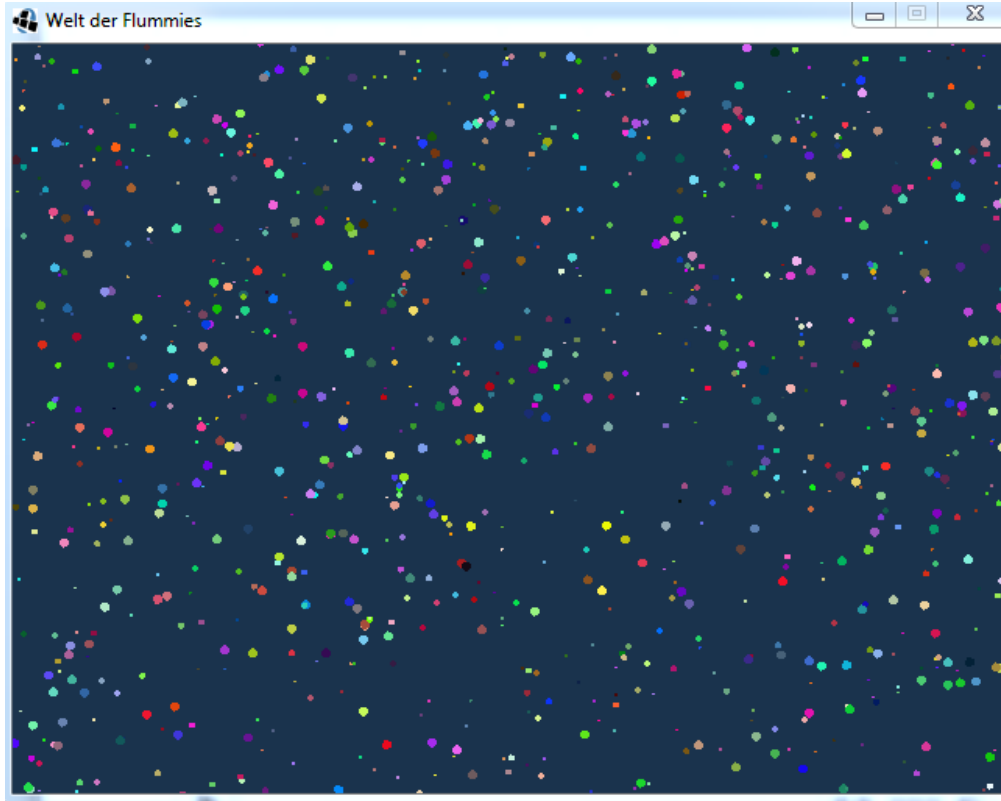
    private void erzeugeFlummies(int anz) {
        Random rand = ThreadLocalRandom.current();
        for (int i = 0; i < anz; i++) {
            flummies.registriereFlummi(new Flummi(rand.nextInt(640),
                rand.nextInt(480), rand.nextFloat()+1, rand.nextInt(3)+1,
                rand.nextFloat(), rand.nextFloat(), rand.nextFloat()));
        }
    }
}
```

```
@Override
public void renderLoop() {
    glEnable(GL_DEPTH_TEST);
    while (!Display.isCloseRequested()) {
        // ... hier OpenGL-Vorbereitungen für Hintergrund
        // löschen usw.

        for (Flummi aktFlummi: flummies.getFlummiMap().values()) {
            aktFlummi.update();
            aktFlummi.render();
        }
        Display.update();
    }
    Display.destroy();
}

public static void main(String[] args) {
    new WeltDerBuntenFlummis().start();
}
}
```

Welt der bunten Flummies



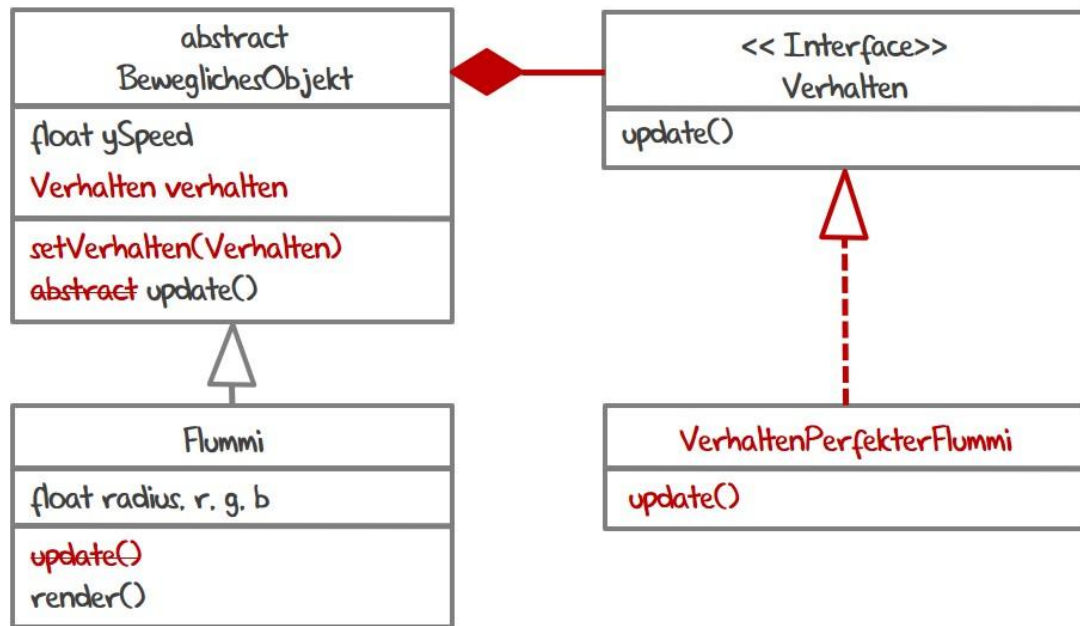
Die Verwaltung sehr vieler Objekte ist jetzt nicht mehr schwierig.



Jetzt wollen wir den Flummies ein
unterschiedliches Verhalten geben

Strategisch mit Verhalten umgehen

Eine „Hat-ein“-Beziehung kann einer „Ist-ein“-Beziehung überlegen sein. Deshalb ist die **Komposition** der Vererbung oft vorzuziehen [1]. Unser Beispiel fällt in dieses Lösungsmuster (Strategy-Pattern). Wir wollen dem Flummi ein Verhalten geben und sollten dieses entsprechend kapseln.



Erweitertes Klassendiagramm
um Verhalten

[1] Gamma E., et al.: "Design Patterns: Entwurfsmuster als Elemente wiederverwendbarer objektorientierter Software", MITP-Verlag, 2015

Das Interface Verhalten

Dazu nutzen wir die Komposition und kapseln das Verhalten. Diese Entkoppelung vom Objekt können wir als **Interface** realisieren:

```
public interface Verhalten {  
    public void update();  
}
```

Das Verhalten eines perfekten Flummis können wir jetzt entsprechend über das Interface implementieren. Schauen wir uns als Beispiel die Implementierung von **VerhaltenPerfekterFlummi** an:

```
public class VerhaltenPerfekterFlummi implements Verhalten {  
    private Flummi flummi;  
  
    public VerhaltenPerfekterFlummi(Flummi flummi) {  
        this.flummi = flummi;  
    }  
}
```

```
@Override  
public void update() {  
    flummi.yPos += flummi.ySpeed;  
    if (flummi.yPos>480 || flummi.yPos<0)  
        flummi.ySpeed *= -1;  
}
```

Komposition in BeweglichesObjekt unterbringen

Jetzt müssen wir die Komposition noch in der Klasse **BeweglichesObjekt** unterbringen und die Ausführung der Methode **update** an das Verhalten weiterdelegieren:

```
public abstract class BeweglichesObjekt extends BasisObjekt {
    public float ySpeed;
    public Verhalten verhalten = null;

    public BeweglichesObjekt(float xPos, float yPos, float ySpeed) {
        super(xPos, yPos);
        this.ySpeed = ySpeed;
    }

    public void setVerhalten(Verhalten verhalten) {
        this.verhalten = verhalten;
    }

    public void update() {
        if (verhalten != null)
            verhalten.update();
    }
}
```

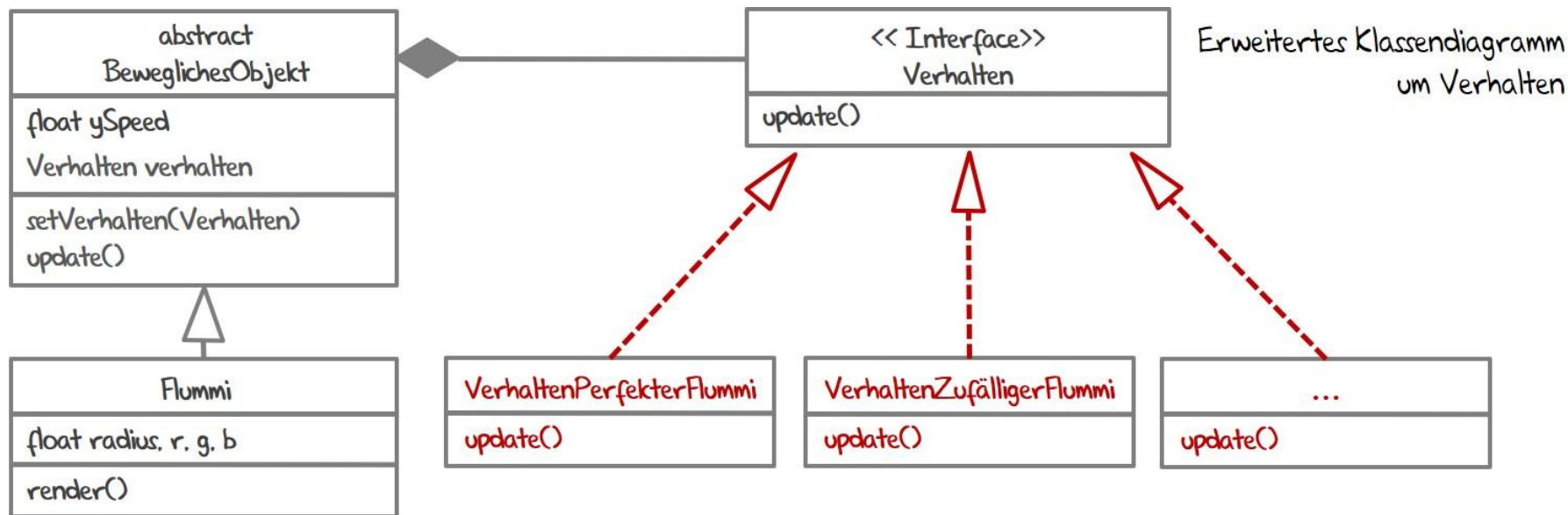
Dieses strategische Konzept, Verhalten zu kapseln, ist sehr wichtig und nützlich. Wir könnten uns sicherlich fragen, warum wir das alles so kompliziert gemacht haben, aber **die Kür kommt jetzt**.



Ab jetzt können wir sehr einfach
das Verhalten der Akteure verändern

Verhalten ganz einfach erweitern

Unsere Objektstruktur steht mit den statischen und dynamischen Objekten bereits fest. Um jetzt das Verhalten der Objekte zu ändern, müssen wir die Klassen aber nicht mehr anfassen, sondern können das durch die [Implementierung des Interface Verhalten](#) vornehmen.



Zufällige Richtungsbewegung realisieren

Fangen wir doch damit an, einem Flummi ein zufälliges Verhalten zu geben und implementieren **VerhaltenZufälligerFlummi**:

```
import java.util.Random;
import java.util.concurrent.ThreadLocalRandom;

public class VerhaltenZufälligerFlummi implements Verhalten {
    private Flummi flummi;
    private Random rand = ThreadLocalRandom.current();

    public VerhaltenZufälligerFlummi(Flummi flummi) {
        this.flummi = flummi;
    }

    @Override
    public void update() {
        flummi.yPos += rand.nextFloat()*2 - 1;
        if (flummi.yPos>480 || flummi.yPos<0)
            flummi.ySpeed *= -1;
    }
}
```

Welt der bunten Flummies erweitern

Jetzt können wir die Spielwelt schon etwas interessanter gestalten. So können wir in der `WeltDerBuntenFlummies` mit der Methode `erzeugeFlummies` bereits drei unterschiedliche Verhaltensmuster erzeugen:

```
private void erzeugeFlummies(int anz) {
    Random rand = ThreadLocalRandom.current();
    for (int i = 0; i < anz; i++) {
        Flummi flummi = new Flummi(rand.nextInt(640), rand.nextInt(480), rand.nextFloat()+1, rand.nextInt(3)+1,
                                   rand.nextFloat(), rand.nextFloat(), rand.nextFloat());

        if (i<anz/3)
            flummi.setVerhalten(new VerhaltenPerfekterFlummi(flummi));
        else if (i>anz/3 && i<2*anz/3)
            flummi.setVerhalten(new VerhaltenZufälligerFlummi(flummi));
        flummies.registriereFlummi(flummi);
    }
}
```

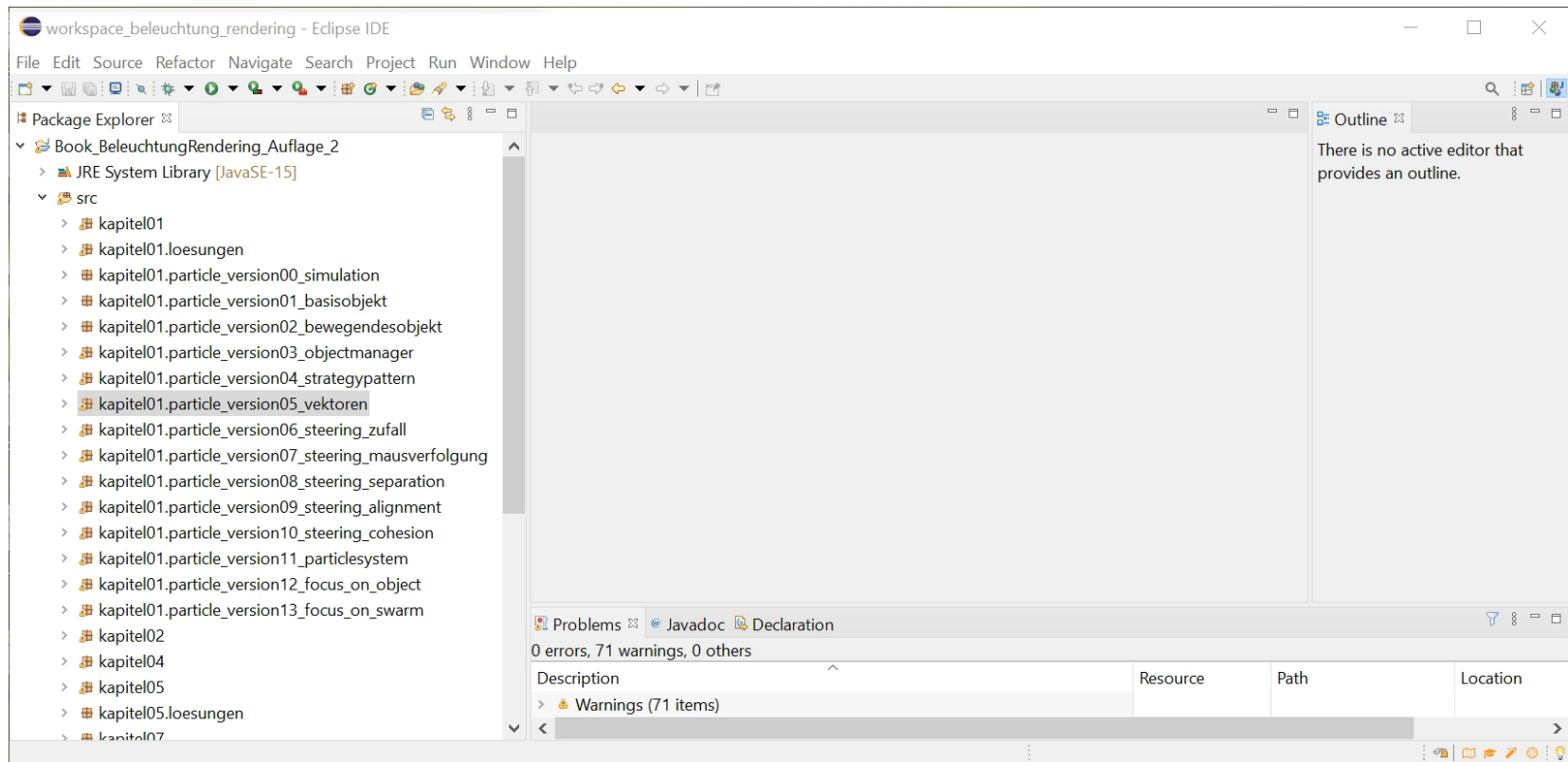
Ein Drittel der Flummies verhält sich **zufällig**, ein Drittel verhält sich **perfekt** und das letzte Drittel erhält **kein Verhalten**, was gleichbedeutend damit ist, dass die Objekte an Ort und Stelle verweilen.

Welt der bunten Flummies erweitert



Die Verwaltung sehr vieler Objekte mit unterschiedlichem Verhalten ist jetzt nicht mehr schwierig.

Programmbeispiele in Eclipse ausprobieren





Analog können wir das einfach in [Processing](#) bauen

Welt der bunten Flummies in Processing

Die Umsetzung in Processing gelingt analog zur Variante in Java.

```
ObjektManager objektManager;  
int globaleObjektID = 1;  
  
void settings() {  
  size(800, 500);  
}  
  
void setup() {  
  surface.setTitle("Welt der bunten Flummies -  
    Processing");  
  objektManager = new ObjektManager();  
  erzeugeFlummies(150);  
}  
  
void draw() {  
  background(25, 55, 90);  
  objektManager.updateAlle();  
  objektManager.renderAlle();  
}  
...
```



Eine kleine Spielumgebung schaffen

Key Takeaways



- Spielumgebung = Kombination aus Rendering + Update
- Trennung von Daten, Verhalten und Darstellung
- Vererbung vs. Komposition
- Strategy Pattern für Verhalten
- Objektverwaltung (Manager / Collections)

Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Erläutern Sie das Singleton-Pattern und warum es sinnvoll ist, den Objektmanager als Singleton zu implementieren.
- (2) Erläutern Sie das Strategie-Pattern. Warum wurde das Verhalten durch eine Komposition gekapselt?



Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

- (1) **Aufgabe 1)** [Java] Erweitern Sie die Welt der bunten Flummies um ein neues Verhalten.



Vielen Dank für die Aufmerksamkeit